MY TAILOR & FABRICS

Tailoring Business Management System

Product Requirements Document

Version 2.1 · FINAL · All Decisions Locked · 2025

Cursor / AI-Agent Ready · Zero Open Questions

Property	Value
Business	My Tailor & Fabrics
Stack	Next.js 14 + TypeScript / NestJS / PostgreSQL / Tailwind CSS / shadcn/ui
Database Host	Managed DB: Supabase / Neon / Railway (connection pooling included)
Scale Target	Multi-branch (2-3 locations) / 24+ employees per branch
Auth	JWT (15min) + Refresh Tokens (7d) / RBAC / 5 roles incl. EMPLOYEE
Search	PostgreSQL tsvector GIN indexes + Redis 30s cache, sub-100ms typeahead
File Storage	Cloudflare R2 (S3-compatible, free egress)
Deployment	Docker + Nginx / VPS (Hetzner / DigitalOcean) / CI/CD GitHub Actions
PRD Version	v2.1 FINAL - all 8 architectural gaps resolved, zero open questions
Money Convention	All amounts stored as INTEGER PAISAS. Rs 1,500 = 150000. Divide by 100 on display.

1. Architectural Decisions — All Gaps Resolved

All 8 previously open questions have been answered. This section documents every decision so there is zero ambiguity when writing code.

Decision 1 — Pricing Model

RESOLVED: Two separate prices per GarmentType set by Admin:

- customerPrice (paisas) — what the shop charges the customer per piece
- employeeRate (paisas) — what the tailor earns per completed piece

These are set globally per garment type. Branches can override both independently.
When creating an order item, both prices are SNAPSHOT-copied onto the OrderItem row (unitPrice + employeeRate) so historical records are never affected by future price changes.

Order Total = SUM(unitPrice × quantity) across all non-cancelled items − discountAmount
Receipt shows: line items, customer prices, discount, total, advance paid, balance due.
Employee earning = SUM(employeeRate × quantity) for COMPLETED/DELIVERED items only.
Cancelled items → employee earns NOTHING (zero pay, no exceptions).

Decision 2 — Branch Pricing

RESOLVED: Global default + branch can override.

GarmentType has: customerPrice (global default) + employeeRate (global default)
BranchGarmentPriceOverride table allows per-branch overrides of either price.
When creating an order item: check for branch override first, fall back to global.
Admin panel shows both global prices and any branch-specific overrides in one screen.

Decision 3 — Item Completion

RESOLVED: All-or-nothing per line item.

An OrderItem with quantity=3 is either PENDING, IN_PROGRESS, COMPLETED, or CANCELLED.
Staff marks the entire line item as completed — no partial qty tracking.
If a customer wants to split (e.g. take 2 now, 1 later), create 2 separate line items.
This keeps the schema simple and the payment calculation unambiguous.

Decision 4 — Overdue Orders

RESOLVED: Auto-change status to OVERDUE + show alert.

A scheduled cron job runs every hour (NestJS @Cron).
Any order with dueDate < NOW() and status NOT IN (DELIVERED, COMPLETED, CANCELLED) is automatically transitioned to OVERDUE status.
OVERDUE orders appear with a red badge in all list views.
Dashboard shows an 'Overdue Orders' KPI card with count + quick-link to list.

OrderStatusHistory logs the auto-transition with actor = 'SYSTEM'.

Decision 5 — Cancelled Order Payment

RESOLVED: No pay for cancelled items. Zero exceptions.

When an order is CANCELLED, ALL its OrderItems are set to status = CANCELLED.
Cancelled items are excluded from employee balance calculation at the DB query level.
No manual override needed — the rule is absolute and enforced in the query.
This is clearly communicated on the Payment screen: 'Earnings from completed items only.'

Decision 6 — Order Discounts

RESOLVED: Both fixed amount AND percentage discounts supported on an order.

Order table has: discountType (FIXED | PERCENTAGE | null), discountValue (Int paisas or %).
Only Admin+ roles can apply a discount.
Calculation:
subtotal = SUM(unitPrice × qty) for non-cancelled items
discountAmount = FIXED: discountValue | PERCENTAGE: FLOOR(subtotal × discountValue / 100)
totalAmount = subtotal − discountAmount
balanceDue = totalAmount − SUM(all OrderPayments)
Receipt shows subtotal, discount line, and final total separately.

Decision 7 — Employee Login

RESOLVED: Employees get their own login (EMPLOYEE role).

Role: EMPLOYEE — lowest privilege level, read-only, scoped to own data only.
Employee login is linked to their Employee record via employeeId field on User.
What EMPLOYEE role can see:

- Their own assigned order items (status, garment type, due date, description)
- Their own payment history and current balance
- Their own profile (read-only)

What EMPLOYEE role CANNOT see:

- Other employees' data, payments, or orders
- Customer details (name, phone, address, measurements)
- Order pricing (unitPrice hidden — only their employeeRate shown)
- Any admin config or management screens

Login via same /auth/login endpoint — role determines dashboard shown.

Decision 8 — Database & Backups

RESOLVED: Managed database (Supabase / Neon / Railway).

Recommended: Neon (PostgreSQL-native, branching for dev/staging, free tier generous)

Alternative: Supabase (adds Auth/Storage if needed later), Railway (simplest DevOps)

Backup strategy (all managed automatically by provider):
- Neon: continuous WAL archiving + point-in-time restore (PITR) up to 7 days
- Supabase Pro: daily backups + PITR
- Railway: daily snapshots

Additional safety: weekly pg_dump export to Cloudflare R2 via GitHub Actions cron.
Connection pooling: use provider's built-in pooler (PgBouncer on Supabase/Neon).
DATABASE_URL in env points to pooled connection string (port 6543 on Supabase).

2. System Architecture

2.1 Architecture Overview

System Layers

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                   CLIENTS                                                        │
│  Next.js 14 App Router · TypeScript · Tailwind CSS · shadcn/ui                                   │
│  Role-based dashboards: SuperAdmin | Admin | Entry | Employee                                    │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │ HTTPS · REST
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────┐
│                              API — NestJS + TypeScript                                           │
│  JwtAuthGuard · RolesGuard · BranchGuard · AuditInterceptor                                      │
│  Modules: auth · branches · customers · employees · orders ·                                     │
│          payments · expenses · config · search · reports                                         │
└──────────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │                                  │
                                  ▼                                  ▼
┌──────────────────────────┐  ┌──────────────────────────┐  ┌──────────────────────┐
│ PostgreSQL (Neon)        │  │ Redis (Upstash)          │  │ CF R2                │
│ Primary data store       │  │ Cache + rate limit       │  │ File store           │
└──────────────────────────┘  └──────────────────────────┘  └──────────────────────┘
```

2.2 Tech Stack (Locked)

Layer	Technology	Notes
Frontend	Next.js 14 + TypeScript (App Router)	Server Components, SSR, file-based routing
UI	Tailwind CSS + shadcn/ui	Component library — copy-paste, fully owned
Backend	NestJS 10 + TypeScript	Modular, DI, decorators, testable
ORM	Prisma 5	Type-safe queries, migrations, Prisma Studio
Database	PostgreSQL 16 via Neon / Supabase	Managed, PITR backups, connection pooling
Cache	Redis via Upstash	Serverless Redis — search cache, rate limiting
Files	Cloudflare R2	S3-compatible, free egress, signed URLs
Auth	JWT + bcrypt (cost 12)	Access 15min · Refresh 7d (HttpOnly cookie)
Validation	class-validator (NestJS DTOs)	Request validation + transformation

Layer	Technology	Notes
Frontend forms	React Hook Form + Zod	Schema validation, type inference
Charts	Recharts	Dashboard visualizations
PDF	@react-pdf/renderer	Order receipts, printable reports
Excel	ExcelJS	Data exports
Scheduling	NestJS @Cron (node-cron)	Hourly overdue order checker
Testing	Jest + Supertest + Playwright	Unit · Integration · E2E
CI/CD	GitHub Actions	Lint → Test → Build → Deploy
Container	Docker + docker-compose	Dev parity + production deploy

2.3 Monorepo Folder Structure

```
tbms/
├── apps/
│   ├── web/                      # Next.js 14 frontend
│   │   ├── app/
│   │   │   ├── (auth)/login/     # Login page
│   │   │   └── (dashboard)/      # Protected routes
│   │   │       ├── layout.tsx    # Sidebar + header shell
│   │   │       ├── page.tsx      # Dashboard (role-aware)
│   │   │       ├── customers/    # List, new, [id]
│   │   │       ├── orders/       # List, new (multi-step), [id]
│   │   │       ├── employees/    # List, new, [id]
│   │   │       ├── payments/     # Employee disbursement
│   │   │       ├── expenses/     # Business expenses
│   │   │       ├── my-orders/    # EMPLOYEE role only
│   │   │       └── admin/        # Config: garment-types, measurements, users, branches
│   │   ├── components/
│   │   │   ├── ui/               # shadcn/ui base components
│   │   │   ├── layout/           # Sidebar, header, nav
│   │   │   ├── orders/           # OrderForm, OrderCard, ItemRow
│   │   │   ├── customers/        # CustomerSearch, MeasurementCard
│   │   │   ├── employees/        # EmployeeCard, PaymentPanel
│   │   │   └── shared/           # StatusBadge, PriceDisplay, DatePicker
│   │   ├── hooks/
│   │   │   ├── useCustomerSearch.ts
│   │   │   ├── useEmployeeSearch.ts
│   │   │   └── useAuth.ts
│   │   └── lib/
│   │       ├── api.ts            # Typed API client (fetch wrapper)
│   │       ├── auth.ts           # Token management
│   │       └── utils.ts          # formatPKR, formatDate, etc.
│   └── api/                      # NestJS backend
```

```
│       ├── src/
│       │   ├── auth/
│       │   ├── branches/
│       │   ├── customers/
│       │   ├── employees/
│       │   ├── orders/
│       │   ├── payments/
│       │   ├── expenses/
│       │   ├── config/
│       │   ├── search/
│       │   ├── reports/
│       │   ├── scheduler/        # Overdue order cron job
│       │   └── common/
│       │       ├── guards/       # JwtAuthGuard, RolesGuard, BranchGuard
│       │       ├── interceptors/ # AuditInterceptor, TransformInterceptor
│       │       ├── filters/      # GlobalExceptionFilter
│       │       └── pipes/        # ParsePaisasPipe, ParseBranchPipe
│       └── prisma/
│           ├── schema.prisma
│           └── migrations/
├── packages/
│   └── shared-types/             # DTOs + interfaces shared by web + api
├── docker-compose.yml
├── docker-compose.prod.yml
└── .github/workflows/ci.yml
```

3. Complete Database Schema (Prisma)

Critical Conventions

1. MONEY: All monetary values stored as INTEGER (paisas). Rs 1,500.00 = 150000.
   Never store floats for money. Divide by 100 for display. Use formatPKR() utility.
2. SOFT DELETE: All main entities have deletedAt DateTime?.
   ALWAYS add WHERE deletedAt IS NULL to queries. Never hard delete.
3. BRANCH SCOPE: Every user-facing entity has branchId.
   BranchGuard auto-applies this filter. Super Admin can override via X-Branch-Id header.
4. SNAPSHOT PRICES: OrderItem stores unitPrice + employeeRate at time of creation.
   This ensures price changes never affect historical orders.
5. AUDIT: AuditInterceptor automatically logs all POST/PUT/PATCH/DELETE calls.

3.1 Enums

```
enum Role            { SUPER_ADMIN ADMIN ENTRY_OPERATOR VIEWER EMPLOYEE }
enum EmployeeStatus  { ACTIVE INACTIVE LEFT }
enum PaymentType     { PER_PIECE WEEKLY_FIXED }
enum CustomerStatus  { ACTIVE INACTIVE BLACKLISTED }
enum FieldType       { NUMBER TEXT DROPDOWN }
enum OrderStatus     { NEW IN_PROGRESS READY OVERDUE DELIVERED COMPLETED CANCELLED }
enum ItemStatus      { PENDING IN_PROGRESS COMPLETED CANCELLED }
enum DiscountType    { FIXED PERCENTAGE }
```

3.2 Branches

```
model Branch {
  id          String     @id @default(cuid())
  code        String     @unique  // e.g. 'LHR', 'KHI' — used in Size Numbers
  name        String
  address     String?
  phone       String?
  isActive    Boolean    @default(true)
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt
  users       User[]
  employees   Employee[]
  customers   Customer[]
  orders      Order[]
  expenses    Expense[]
  priceOverrides BranchGarmentPrice[]
}
```

3.3 Users & Auth

```
model User {
  id            String    @id @default(cuid())
  name          String
```

```
  email         String    @unique
  passwordHash  String
  role          Role      @default(ENTRY_OPERATOR)
  branchId      String?   // null = Super Admin (all branches)
  branch        Branch?   @relation(fields: [branchId], references: [id])
  employeeId    String?   @unique  // only set when role = EMPLOYEE
  employee      Employee? @relation(fields: [employeeId], references: [id])
  isActive      Boolean   @default(true)
  lastLoginAt   DateTime?
  refreshToken  String?   // hashed refresh token
  createdAt     DateTime  @default(now())
  updatedAt     DateTime  @updatedAt
  deletedAt     DateTime?
  auditLogs     AuditLog[]
  paymentsProcessed Payment[]
  @@index([branchId])
  @@index([role])
}
```

3.4 Employees

```
model Employee {
  id              String         @id @default(cuid())
  employeeCode    String         @unique  // auto-gen: EMP-LHR-0001
  branchId        String
  branch          Branch         @relation(fields: [branchId], references: [id])
  fullName        String
  fatherName      String?
  phone           String
  phone2          String?
  address         String?
  city            String?
  cnic            String?        @unique
  dateOfBirth     DateTime?
  dateOfJoining   DateTime       @default(now())
  designation     String?
  paymentType     PaymentType    @default(PER_PIECE)
  accountNumber   String?        // bank / JazzCash / EasyPaisa
  emergencyName   String?
  emergencyPhone  String?
  photoUrl        String?
  status          EmployeeStatus @default(ACTIVE)
  notes           String?
  searchVector    Unsupported("tsvector")?
  createdAt       DateTime       @default(now())
  updatedAt       DateTime       @updatedAt
  deletedAt       DateTime?
  documents       EmployeeDocument[]
  orderItems      OrderItem[]
  payments        Payment[]
  userAccount     User?          // linked login account (EMPLOYEE role)
  @@index([branchId, status])
  @@index([searchVector], type: Gin)
```

```
}
```

```
model EmployeeDocument {
  id           String   @id @default(cuid())
  employeeId   String
  employee     Employee @relation(fields: [employeeId], references: [id])
  label        String   // e.g. 'CNIC Front', 'Contract'
  fileUrl      String   // Cloudflare R2 signed URL key
  fileType     String   // 'image/jpeg', 'application/pdf'
  uploadedById String
  createdAt    DateTime @default(now())
  @@index([employeeId])
}
```

3.5 Customers & Measurements

```
model Customer {
  id            String         @id @default(cuid())
  branchId      String
  branch        Branch         @relation(fields: [branchId], references: [id])
  sizeNumber    String         @unique  // C-LHR-0045 — critical fast-lookup field
  fullName      String
  phone         String
  whatsapp      String?
  email         String?
  address       String?
  city          String?
  notes         String?
  status        CustomerStatus @default(ACTIVE)
  searchVector  Unsupported("tsvector")?
  createdAt     DateTime       @default(now())
  updatedAt     DateTime       @updatedAt
  deletedAt     DateTime?
  measurements  CustomerMeasurement[]
  orders        Order[]
  @@index([branchId, deletedAt])
  @@index([sizeNumber])           // prefix search for order entry
  @@index([searchVector], type: Gin)
}
```

```
model MeasurementCategory {
  id          String   @id @default(cuid())
  name        String   // 'Shalwar Kameez', 'Coat', 'Waistcoat'
  sortOrder   Int      @default(0)
  isActive    Boolean  @default(true)
  createdAt   DateTime @default(now())
  fields      MeasurementField[]
  measurements CustomerMeasurement[]
}
```

```
model MeasurementField {
  id          String              @id @default(cuid())
```

```
  categoryId  String
  category    MeasurementCategory @relation(fields: [categoryId], references: [id])
  label       String              // 'Shoulder', 'Chest', 'Collar'
  fieldType   FieldType           @default(NUMBER)
  unit        String?             // 'inches', 'cm'
  isRequired  Boolean             @default(false)
  sortOrder   Int                 @default(0)
  dropdownOptions String[]        // only used when fieldType = DROPDOWN
}
```

```
model CustomerMeasurement {
  id          String              @id @default(cuid())
  customerId  String
  customer    Customer            @relation(fields: [customerId], references: [id])
  categoryId  String
  category    MeasurementCategory @relation(fields: [categoryId], references: [id])
  // values: { "fieldId_abc": "42", "fieldId_def": "16.5" }
  values      Json
  updatedAt   DateTime            @updatedAt
  @@unique([customerId, categoryId])  // one measurement set per category per customer
}
```

3.6 Garment Types & Branch Price Overrides

```
model GarmentType {
  id             String   @id @default(cuid())
  name           String   // 'Shalwar Kameez', '3-Piece Suit', 'Waistcoat'
  customerPrice  Int      // paisas — global default: what shop charges customer
  employeeRate   Int      // paisas — global default: what tailor earns per piece
  description    String?
  sortOrder      Int      @default(0)
  isActive       Boolean  @default(true)
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
  orderItems     OrderItem[]
  branchOverrides BranchGarmentPrice[]
}
```

```
// Per-branch price overrides (Decision 2)
model BranchGarmentPrice {
  id             String      @id @default(cuid())
  branchId       String
  branch         Branch      @relation(fields: [branchId], references: [id])
  garmentTypeId  String
  garmentType    GarmentType @relation(fields: [garmentTypeId], references: [id])
  customerPrice  Int?        // null = use global default
  employeeRate   Int?        // null = use global default
  updatedAt      DateTime    @updatedAt
  @@unique([branchId, garmentTypeId])
}
```

3.7 Orders & Items

```
model Order {
  id              String        @id @default(cuid())
  orderNumber     String        @unique  // ORD-2025-LHR-00047
  branchId        String
  branch          Branch        @relation(fields: [branchId], references: [id])
  customerId      String
  customer        Customer      @relation(fields: [customerId], references: [id])
  orderDate       DateTime      @default(now())
  dueDate         DateTime
  subtotal        Int           // paisas: SUM(unitPrice × qty) non-cancelled items
  discountType    DiscountType? // FIXED | PERCENTAGE | null
  discountValue   Int           @default(0)  // paisas if FIXED, basis points if PERCENTAGE
  discountAmount  Int           @default(0)  // paisas: computed and stored
  totalAmount     Int           // paisas: subtotal – discountAmount
  totalPaid       Int           @default(0)  // paisas: SUM of all OrderPayments
  balanceDue      Int           // paisas: totalAmount – totalPaid (denormalized)
  notes           String?
  status          OrderStatus   @default(NEW)
  createdById     String
  searchVector    Unsupported("tsvector")?
  createdAt       DateTime      @default(now())
  updatedAt       DateTime      @updatedAt
  deletedAt       DateTime?
  items           OrderItem[]
  payments        OrderPayment[]
  statusHistory   OrderStatusHistory[]
  @@index([branchId, status])
  @@index([customerId])
  @@index([dueDate])            // overdue cron query
  @@index([searchVector], type: Gin)
}
```

```
model OrderItem {
  id              String      @id @default(cuid())
  orderId         String
  order           Order       @relation(fields: [orderId], references: [id])
  garmentTypeId   String
  garmentType     GarmentType @relation(fields: [garmentTypeId], references: [id])
  garmentTypeName String      // SNAPSHOT: name at time of order
  employeeId      String?
  employee        Employee?   @relation(fields: [employeeId], references: [id])
  quantity        Int         @default(1)
  unitPrice       Int         // paisas: SNAPSHOT customer price at order time
  employeeRate    Int         // paisas: SNAPSHOT employee rate at order time
  description     String?     // fabric colour, embroidery, button type, etc.
  dueDate         DateTime?   // item-level due date (can differ from order)
  status          ItemStatus  @default(PENDING)
  completedAt     DateTime?   // set when status → COMPLETED
  @@index([orderId])
  @@index([employeeId, status])  // employee earnings query
}
```

```
model OrderPayment {
  id          String   @id @default(cuid())
  orderId     String
  order       Order    @relation(fields: [orderId], references: [id])
  amount      Int      // paisas received from customer
  paidAt      DateTime @default(now())
  receivedById String  // userId
  note        String?
  @@index([orderId])
}
```

```
model OrderStatusHistory {
  id         String      @id @default(cuid())
  orderId    String
  order      Order       @relation(fields: [orderId], references: [id])
  fromStatus OrderStatus?
  toStatus   OrderStatus
  changedById String?    // null when actor = SYSTEM (cron job)
  actor      String      @default("USER")  // 'USER' | 'SYSTEM'
  note       String?
  createdAt  DateTime    @default(now())
  @@index([orderId])
}
```

3.8 Payments, Expenses & Audit

```
model Payment {
  id           String   @id @default(cuid())
  employeeId   String
  employee     Employee @relation(fields: [employeeId], references: [id])
  amount       Int      // paisas disbursed to employee
  paidAt       DateTime @default(now())
  processedById String
  processedBy  User     @relation(fields: [processedById], references: [id])
  note         String?
  @@index([employeeId, paidAt])
}
```

```
model ExpenseCategory {
  id       String    @id @default(cuid())
  name     String    // 'Rent', 'Electricity', 'Equipment', 'Supplies'
  isActive Boolean   @default(true)
  expenses Expense[]
}
```

```
model Expense {
  id          String          @id @default(cuid())
  branchId    String
  branch      Branch          @relation(fields: [branchId], references: [id])
  categoryId  String
  category    ExpenseCategory @relation(fields: [categoryId], references: [id])
  amount      Int             // paisas
```

```
  description String?
  receiptUrl  String?
  expenseDate DateTime
  addedById   String
  createdAt   DateTime        @default(now())
  @@index([branchId, expenseDate])
}
```

```
model AuditLog {
  id        String   @id @default(cuid())
  userId    String
  user      User     @relation(fields: [userId], references: [id])
  action    String   // CREATE | UPDATE | DELETE | LOGIN | PAYMENT | STATUS_CHANGE
  entity    String   // 'Order' | 'Employee' | 'Customer' | 'Payment' ...
  entityId  String
  branchId  String?
  oldValue  Json?    // snapshot before mutation
  newValue  Json?    // snapshot after mutation
  ipAddress String?
  userAgent String?
  createdAt DateTime @default(now())
  @@index([entity, entityId])
  @@index([userId])
  @@index([createdAt])
}
```

4. Search Optimization

Search must feel instant. Staff search customers tens of times per hour during order entry. Target: sub-100ms P95 latency on all typeahead endpoints.

4.1 PostgreSQL Full-Text Search — Migrations

```
-- Run after initial schema migration

-- CUSTOMERS: search by name + size number + phone
ALTER TABLE customers ADD COLUMN IF NOT EXISTS search_vector tsvector
  GENERATED ALWAYS AS (
    to_tsvector('simple',
      coalesce(full_name, '')    || ' ' ||
      coalesce(size_number, '')  || ' ' ||
      coalesce(phone, '')        || ' ' ||
      coalesce(whatsapp, '')     || ' ' ||
      coalesce(city, '')
    )
  ) STORED;
CREATE INDEX IF NOT EXISTS customers_fts_idx ON customers USING GIN(search_vector);

-- EMPLOYEES: search by name + code + phone + cnic
ALTER TABLE employees ADD COLUMN IF NOT EXISTS search_vector tsvector
  GENERATED ALWAYS AS (
    to_tsvector('simple',
      coalesce(full_name, '')      || ' ' ||
      coalesce(employee_code, '')  || ' ' ||
      coalesce(phone, '')          || ' ' ||
      coalesce(cnic, '')           || ' ' ||
      coalesce(designation, '')
    )
  ) STORED;
CREATE INDEX IF NOT EXISTS employees_fts_idx ON employees USING GIN(search_vector);
```

4.2 Search Service (NestJS)

```
// src/search/search.service.ts
@Injectable()
export class SearchService {
  constructor(private prisma: PrismaService, private cache: CacheService) {}

  async searchCustomers(query: string, branchId: string, limit = 10) {
    if (!query || query.trim().length < 2) return [];
    const q = query.trim();
    const cacheKey = `search:cust:${branchId}:${q}`;
    const hit = await this.cache.get<Customer[]>(cacheKey);
    if (hit) return hit;

    // Fast path: looks like a Size Number
```

```
    if (/^C-/i.test(q)) {
      const result = await this.prisma.$queryRaw<Customer[]>`
        SELECT id, full_name, size_number, phone, city, status
        FROM customers
        WHERE branch_id = ${branchId} AND deleted_at IS NULL
          AND size_number ILIKE ${q + '%'}
        LIMIT ${limit}`;
      await this.cache.set(cacheKey, result, 30);
      return result;
    }

    // Full-text search
    const result = await this.prisma.$queryRaw<Customer[]>`
      SELECT id, full_name, size_number, phone, city, status,
             ts_rank(search_vector, plainto_tsquery('simple', ${q})) AS rank
      FROM customers
      WHERE branch_id = ${branchId} AND deleted_at IS NULL
        AND search_vector @@ plainto_tsquery('simple', ${q})
      ORDER BY rank DESC
      LIMIT ${limit}`;

    await this.cache.set(cacheKey, result, 30); // 30s TTL
    return result;
  }

  async searchEmployees(query: string, branchId: string, limit = 10) {
    if (!query || query.trim().length < 2) return [];
    const q = query.trim();
    const cacheKey = `search:emp:${branchId}:${q}`;
    const hit = await this.cache.get<Employee[]>(cacheKey);
    if (hit) return hit;

    const result = await this.prisma.$queryRaw<Employee[]>`
      SELECT id, full_name, employee_code, designation, status
      FROM employees
      WHERE branch_id = ${branchId} AND deleted_at IS NULL AND status = 'ACTIVE'
        AND search_vector @@ plainto_tsquery('simple', ${q})
      ORDER BY ts_rank(search_vector, plainto_tsquery('simple', ${q})) DESC
      LIMIT ${limit}`;

    await this.cache.set(cacheKey, result, 30);
    return result;
  }
}
```

4.3 Frontend Debounced Search Hook

```
// hooks/useCustomerSearch.ts
export function useCustomerSearch(branchId: string) {
  const [query, setQuery]     = useState('');
  const [results, setResults] = useState<CustomerSearchResult[]>([]);
  const [loading, setLoading] = useState(false);
```

```
  useEffect(() => {
    if (query.trim().length < 2) { setResults([]); return; }
    const timer = setTimeout(async () => {
      setLoading(true);
      try {
        const data = await api.get(`/search/customers?q=${encodeURIComponent(query)}`);
        setResults(data);
      } finally { setLoading(false); }
    }, 250); // 250ms debounce — feels instant, avoids API spam
    return () => clearTimeout(timer);
  }, [query, branchId]);

  return { query, setQuery, results, loading };
}
```

4.4 Redis Caching Strategy

Cache Key	TTL	Invalidate When
search:cust:{branchId}:{q}	30s	Any customer created/updated in branch
search:emp:{branchId}:{q}	30s	Any employee created/updated in branch
garment_types	5min	Admin adds/edits/disables garment type
measure_categories	5min	Admin changes measurement config
branch_prices:{branchId}	5min	Admin changes branch price override
dashboard:{branchId}:{weekKey}	5min	New order / payment / expense in branch
emp_balance:{employeeId}	60s	New payment processed or item completed

4.5 Performance Targets

Endpoint	Target P95	Strategy
Customer typeahead /search/customers	< 80ms	GIN index + size-number fast path + Redis 30s
Employee typeahead /search/employees	< 80ms	GIN index + Redis 30s
Order list page 1	< 150ms	Composite indexes + pagination (20/page)
Employee balance calculation	< 200ms	Redis 60s + optimised raw SQL
Dashboard KPI cards	< 300ms	Redis 5min per branch
PDF receipt generation	< 2s	@react-pdf with template reuse
Excel export (1000 rows)	< 3s	Streamed ExcelJS response

5. Core Business Logic

5.1 Price Resolution (Branch Override)

```
// src/config/price-resolver.service.ts
async resolvePrices(garmentTypeId: string, branchId: string) {
  const [global, override] = await Promise.all([
    this.prisma.garmentType.findUniqueOrThrow({ where: { id: garmentTypeId } }),
    this.prisma.branchGarmentPrice.findUnique({
      where: { branchId_garmentTypeId: { branchId, garmentTypeId } }
    }),
  ]);
  return {
    customerPrice: override?.customerPrice ?? global.customerPrice,
    employeeRate:  override?.employeeRate  ?? global.employeeRate,
  };
}
```

5.2 Order Total Recalculation

```
// Called after any OrderItem is added, updated, or cancelled
// Also called after discount is applied
async recalcOrderTotals(orderId: string, tx?: PrismaTransaction) {
  const db = tx ?? this.prisma;
  const items = await db.orderItem.findMany({
    where: { orderId, status: { not: 'CANCELLED' } },
    select: { unitPrice: true, quantity: true },
  });
  const subtotal = items.reduce((sum, i) => sum + i.unitPrice * i.quantity, 0);

  const order = await db.order.findUniqueOrThrow({
    where: { id: orderId },
    select: { discountType: true, discountValue: true, totalPaid: true },
  });

  let discountAmount = 0;
  if (order.discountType === 'FIXED') {
    discountAmount = Math.min(order.discountValue, subtotal); // never go negative
  } else if (order.discountType === 'PERCENTAGE') {
    discountAmount = Math.floor(subtotal * order.discountValue / 100);
  }

  const totalAmount = subtotal - discountAmount;
  const balanceDue  = Math.max(0, totalAmount - order.totalPaid);

  await db.order.update({
    where: { id: orderId },
    data: { subtotal, discountAmount, totalAmount, balanceDue },
  });
}
```

5.3 Employee Balance Calculation

```
// src/payments/payment.service.ts
async getEmployeeBalanceSummary(employeeId: string) {
  const [earnedResult, totalPaid] = await Promise.all([
    // Raw SQL: SUM(employeeRate * quantity) for all COMPLETED/DELIVERED items
    this.prisma.$queryRaw<[{ total: BigInt }]>`
      SELECT COALESCE(SUM(employee_rate * quantity), 0) AS total
      FROM order_items
      WHERE employee_id = ${employeeId}
        AND status IN ('COMPLETED', 'DELIVERED')`,
    this.prisma.payment.aggregate({
      where: { employeeId },
      _sum: { amount: true },
    }),
  ]);

  const totalEarned = Number(earnedResult[0].total);
  const paid        = totalPaid._sum.amount ?? 0;
  const balance     = totalEarned - paid;

  // Validate: balance should never go negative (guard in disbursement)
  return {
    totalEarned,          // all-time
    totalPaid: paid,      // all-time
    currentBalance: balance,
    weekly: await this.getWeeklyBreakdown(employeeId),
  };
}
```

```
async disbursePay(employeeId: string, amount: number, processedById: string, note?: string) {
  const summary = await this.getEmployeeBalanceSummary(employeeId);
  if (amount > summary.currentBalance) {
    throw new UnprocessableEntityException({
      code: 'PAYMENT_EXCEEDS_BALANCE',
      message: `Cannot disburse ${formatPKR(amount)}. Balance is
${formatPKR(summary.currentBalance)}`,
    });
  }
  return this.prisma.payment.create({
    data: { employeeId, amount, processedById, note },
  });
}
```

5.4 Weekly Breakdown (PKT Timezone)

```
async getWeeklyBreakdown(employeeId: string, weeksBack = 8) {
  // Week = Monday 00:00 → Sunday 23:59 in Asia/Karachi (PKT = UTC+5)
  return this.prisma.$queryRaw`
    WITH weekly_earned AS (
      SELECT
        date_trunc('week', completed_at AT TIME ZONE 'Asia/Karachi') AS week_start,
        SUM(employee_rate * quantity) AS earned
```

```
      FROM order_items
      WHERE employee_id = ${employeeId}
        AND status IN ('COMPLETED', 'DELIVERED')
        AND completed_at >= NOW() - INTERVAL '${weeksBack} weeks'
      GROUP BY week_start
    ),
    weekly_paid AS (
      SELECT
        date_trunc('week', paid_at AT TIME ZONE 'Asia/Karachi') AS week_start,
        SUM(amount) AS paid
      FROM payments
      WHERE employee_id = ${employeeId}
        AND paid_at >= NOW() - INTERVAL '${weeksBack} weeks'
      GROUP BY week_start
    )
    SELECT
      we.week_start,
      we.week_start + INTERVAL '6 days' AS week_end,
      COALESCE(we.earned, 0) AS earned,
      COALESCE(wp.paid, 0)   AS paid,
      COALESCE(we.earned, 0) - COALESCE(wp.paid, 0) AS closing_balance
    FROM weekly_earned we
    LEFT JOIN weekly_paid wp USING (week_start)
    ORDER BY week_start DESC
  `;
}
```

5.5 Overdue Order Cron Job

```
// src/scheduler/overdue.scheduler.ts
@Injectable()
export class OverdueScheduler {
  constructor(private prisma: PrismaService) {}

  @Cron(CronExpression.EVERY_HOUR)
  async markOverdueOrders() {
    const now = new Date();
    const result = await this.prisma.order.updateMany({
      where: {
        dueDate: { lt: now },
        status:  { in: ['NEW', 'IN_PROGRESS', 'READY'] },
        deletedAt: null,
      },
      data: { status: 'OVERDUE' },
    });

    if (result.count > 0) {
      // Log to audit with actor = SYSTEM
      await this.logOverdueTransitions(result.count);
    }
  }
}
```

5.6 Order Status Transition Rules

From	To	Min Role	Condition / Rule
NEW	IN_PROGRESS	Entry+	At least one OrderItem has employeeId assigned
IN_PROGRESS	READY	Entry+	ALL non-cancelled OrderItems are COMPLETED
READY	DELIVERED	Entry+	Manual confirmation (customer collected order)
DELIVERED	COMPLETED	Entry+	balanceDue = 0 (full payment received)
Any active	OVERDUE	SYSTEM	Automatic: dueDate < NOW() and status not terminal
Any	CANCELLED	Admin+	Reason required. ALL items set to CANCELLED. Employee earns nothing.
OVERDUE	IN_PROGRESS	Entry+	Manual reset (e.g. new due date set)

6. API Reference

6.1 Conventions

```
Base URL:       https://api.domain.com/api/v1
Auth:           Authorization: Bearer <access_token>
Branch scope:   X-Branch-Id: <branchId>  (required for non-Super-Admin)
Money:          All amounts in request/response are INTEGER PAISAS
Pagination:     ?page=1&limit=20&sortBy=createdAt&sortDir=desc

// Standard success envelope
{ "success": true, "data": {...}, "meta": { "page": 1, "total": 145 } }
// Standard error envelope
{ "success": false, "error": { "code": "PAYMENT_EXCEEDS_BALANCE", "message": "...", "statusCode":
422 } }
```

6.2 Full Endpoint List

Method	Path	Min Role	Notes
POST	/auth/login	Public	{ email, password } → { accessToken, user }
POST	/auth/refresh	Public	Cookie refresh token → new accessToken
POST	/auth/logout	Any	Invalidate refresh token
GET	/auth/me	Any	Current user + branch info
GET	/search/customers?q=&limit=	Entry+	FTS typeahead. Returns max 10.
GET	/search/employees?q=&limit=	Entry+	FTS typeahead. Active only.
GET	/customers	Entry+	?search= ?status= ?page= ?limit=
POST	/customers	Entry+	Creates + auto-generates sizeNumber
GET	/customers/:id	Entry+	Full profile + measurements + order summary
PUT	/customers/:id	Entry+	Update info (not sizeNumber — immutable)
DELETE	/customers/:id	Admin+	Soft delete
GET	/customers/:id/orders	Entry+	Paginated order history
POST	/customers/:id/measurements	Entry+	Upsert measurement category (JSON values)
GET	/employees	Admin+	?search= ?status= ?page= ?limit=
POST	/employees	Admin+	Create employee

Method	Path	Min Role	Notes
GET	/employees/:id	Admin+	Full profile with work + payment summary
PUT	/employees/:id	Admin+	Update employee info
DELETE	/employees/:id	Admin+	Soft delete (archive)
POST	/employees/:id/documents	Admin+	multipart/form-data upload to R2
DELETE	/employees/:id/documents/:docId	Admin+	Delete document
POST	/employees/:id/user-account	Admin+	Create EMPLOYEE login account
GET	/orders	Entry+	?search= ?status= ?employeeId= ?from= ?to=
POST	/orders	Entry+	Create order with items array in body
GET	/orders/:id	Entry+	Full order + items + payments + history
PUT	/orders/:id	Entry+	Update dueDate, notes, status, discount
DELETE	/orders/:id	Admin+	Soft cancel (sets status = CANCELLED)
POST	/orders/:id/items	Entry+	Add item (resolves prices from branch)
PUT	/orders/:id/items/:itemId	Entry+	Update status, employee, description
DELETE	/orders/:id/items/:itemId	Admin+	Cancel item (soft)
POST	/orders/:id/payments	Entry+	Record customer payment
GET	/orders/:id/receipt	Entry+	PDF receipt (binary response)
GET	/payments/employee/:id/summary	Admin+	Balance, weekly breakdown, all-time stats
POST	/payments	Admin+	{ employeeId, amount, note } — disburse
GET	/payments/employee/:id/history	Admin+	?from= ?to= paginated ledger
GET	/payments/weekly-report	Admin+	All employees this week — printable summary
GET	/expenses	Admin+	?categoryId= ?from= ?to= ?branchId=
POST	/expenses	Admin+	Create expense + optional receipt upload
PUT	/expenses/:id	Admin+	Edit expense
DELETE	/expenses/:id	Admin+	Soft delete
GET	/config/garment-types	Any	Active types with resolved branch prices

Method	Path	Min Role	Notes
POST	/config/garment-types	Admin+	Create type with customerPrice + employeeRate
PUT	/config/garment-types/:id	Admin+	Edit global prices / name / status
GET	/config/garment-types/:id/branch-prices	Admin+	All branch overrides for this type
PUT	/config/garment-types/:id/branch-prices	Admin+	Set branch override (null = use global)
GET	/config/measurement-categories	Any	Active categories with fields
POST	/config/measurement-categories	Admin+	Create category
POST	/config/measurement-categories/:id/fields	Admin+	Add field to category
PUT	/config/measurement-fields/:id	Admin+	Edit field label, type, options
DELETE	/config/measurement-fields/:id	Admin+	Soft delete field
GET	/reports/dashboard	Admin+	KPIs + chart data for current branch
GET	/reports/orders	Admin+	Export ?format=pdf|excel ?from= ?to=
GET	/reports/payments	Admin+	Export payment summary
GET	/reports/expenses	Admin+	Export expense breakdown
GET	/my/orders	Employee	Own assigned order items only
GET	/my/payments	Employee	Own payment history + balance (rate only, no customer prices)
GET	/my/profile	Employee	Own profile (read-only)

7. Roles & Permissions

7.1 Role Definitions (5 Roles)

Role	Who	Branch Scope	Summary
SUPER_ADMIN	Business Owner	All branches	Full access. Can do everything. Can switch branches via X-Branch-Id header.
ADMIN	Shop Manager	Own branch only	Manage employees, customers, orders, payments, expenses, config. Cannot manage users or branches.
ENTRY_OPERATOR	Front desk staff	Own branch only	Create/edit customers and orders, record payments, view reports. Cannot delete or manage config.
VIEWER	Accountant / read-only	Own branch only	View orders, reports, payment history. No create/edit/delete.
EMPLOYEE	Tailor (own data only)	Own branch only	View own assigned order items, own payment history, own profile. No customer data, no pricing.

7.2 Permission Matrix

Action	SA	Admin	Entry	Viewer	Emp
Manage user accounts	✓	✗	✗	✗	✗
Manage branches	✓	✗	✗	✗	✗
Configure garment types + prices	✓	✓	✗	✗	✗
Configure measurement categories	✓	✓	✗	✗	✗
Set branch price overrides	✓	✓	✗	✗	✗
Create / edit employees	✓	✓	✗	✗	✗
Upload employee documents	✓	✓	✗	✗	✗
Create employee login account	✓	✓	✗	✗	✗
Create / edit customers	✓	✓	✓	✗	✗
Add / edit measurements	✓	✓	✓	✗	✗
Create orders	✓	✓	✓	✗	✗
Update order status	✓	✓	✓	✗	✗
Cancel orders (soft delete)	✓	✓	✗	✗	✗
Apply discounts to orders	✓	✓	✗	✗	✗

Action	SA	Admin	Entry	Viewer	Emp
Record customer payment	✓	✓	✓	✗	✗
Disburse employee payment	✓	✓	✗	✗	✗
Add business expenses	✓	✓	✗	✗	✗
View reports & dashboard	✓	✓	✗	✓	✗
Export PDF / Excel reports	✓	✓	✗	✓	✗
View own assigned orders	✓	✓	✓	✗	✓
View own payment balance	✓	✓	✗	✗	✓
View own profile	✓	✓	✗	✗	✓
See customer prices (unitPrice)	✓	✓	✓	✓	✗

8. Security & Scalability

8.1 Security Checklist

Requirement	Implementation	Priority
Password hashing	bcrypt cost factor 12	P0
JWT storage	Access token in memory (not localStorage). Refresh token in HttpOnly cookie.	P0
RBAC enforcement	RolesGuard + BranchGuard on every route. Applied globally in AppModule.	P0
Branch isolation	BranchGuard injects req.branchId. All service methods receive and filter by it.	P0
Soft delete only	deletedAt field on all entities. Hard delete not exposed anywhere.	P0
SQL injection	Prisma parameterized queries only. Never string-concatenate SQL.	P0
Rate limiting	ThrottlerModule: 60 req/min global. 5/min on /auth/login.	P0
CORS	Whitelist frontend domain only. No wildcard.	P0
Security headers	NestJS Helmet middleware. Sets CSP, HSTS, X-Frame-Options, etc.	P0
File upload	Validate MIME type server-side (file-type package). Max 10MB. R2 private bucket.	P0
Audit logging	AuditInterceptor on all POST/PUT/PATCH/DELETE routes. Actor + before/after.	P1
Sensitive action confirm	Delete and disbursement require admin password re-entry in frontend.	P1
Employee data isolation	EMPLOYEE role: API enforces employeeId === req.user.employeeId on all /my/ routes.	P0
Refresh token rotation	Rotate on every refresh. Invalidate old token. Store hashed in DB.	P1
HTTPS	Nginx with Let's Encrypt SSL. HTTP → HTTPS redirect.	P0

8.2 Multi-Branch Scalability

Scaling Strategy

Phase 1–3 (1–3 branches): Single PostgreSQL instance on managed DB (Neon/Supabase). All tables have branch_id. Queries scoped per branch. No schema changes needed.

Phase 4+ (5–10 branches): Enable Neon branching or Supabase read replicas.
Add Redis cluster. Move to dedicated connection pooler (PgBouncer).

Phase 5+ (10+ branches / franchise): Migrate to schema-per-tenant PostgreSQL.
Use the same codebase — only the DATABASE_URL changes per tenant.
This is a planned migration path, not a rewrite.

8.3 Key Database Indexes Summary

Table	Columns	Type	Query it serves
customers	search_vector	GIN	FTS typeahead search
customers	size_number	BTREE	Exact / prefix order lookup
customers	(branch_id, deleted_at)	BTREE	Branch-scoped list
employees	search_vector	GIN	FTS typeahead search
employees	(branch_id, status)	BTREE	Active employees in branch
orders	(branch_id, status)	BTREE	Orders by status in branch
orders	customer_id	BTREE	Customer order history
orders	due_date	BTREE	Overdue cron: dueDate < NOW()
order_items	(employee_id, status)	BTREE	Employee earnings calculation
order_items	order_id	BTREE	Eager load items for order
payments	(employee_id, paid_at)	BTREE	Weekly breakdown query
expenses	(branch_id, expense_date)	BTREE	Monthly expense reports
audit_logs	(entity, entity_id)	BTREE	Record change history

8.4 Environment Variables

```
# .env.example — ALL required vars

# Database (Neon pooled connection)
DATABASE_URL=postgresql://user:pass@ep-xxx.us-east-2.aws.neon.tech:6543/tbms?pgbouncer=true
DIRECT_URL=postgresql://user:pass@ep-xxx.us-east-2.aws.neon.tech:5432/tbms
# Note: Prisma needs DIRECT_URL for migrations, DATABASE_URL for queries

# Redis (Upstash)
REDIS_URL=rediss://default:token@endpoint.upstash.io:6380

# JWT
JWT_SECRET=<256-bit-random-string>
```

```
JWT_EXPIRES_IN=15m
JWT_REFRESH_SECRET=<different-256-bit-random-string>
JWT_REFRESH_EXPIRES_IN=7d

# Cloudflare R2
R2_ACCOUNT_ID=
R2_ACCESS_KEY_ID=
R2_SECRET_ACCESS_KEY=
R2_BUCKET=tbms-files
R2_PUBLIC_URL=https://files.yourdomain.com

# App
NODE_ENV=production
PORT=3001
FRONTEND_URL=https://yourdomain.com
TZ=Asia/Karachi
```

9. Frontend Specifications

9.1 Role-Based Dashboard Routing

```
// middleware.ts — redirect based on role on login
SUPER_ADMIN  → /dashboard        (full dashboard)
ADMIN        → /dashboard        (full dashboard, branch-scoped)
ENTRY_OPERATOR → /orders         (direct to orders list)
VIEWER       → /reports          (direct to reports)
EMPLOYEE     → /my-orders        (own assigned work only)
```

9.2 Order Creation — Multi-Step Form

Step	Title	What Happens
1	Customer	Search bar with 250ms debounce typeahead. Select customer → shows preview card (name, size number, last order). Or create new customer inline.
2	Items	Add garment line items. Each item: GarmentType picker (shows customerPrice) → Employee picker (typeahead) → Qty input → Description textarea → Item due date. Can add unlimited items.
3	Payment	Shows: subtotal auto-calculated. Optional discount (toggle FIXED/PERCENTAGE → input value). Final total. Advance payment input. Remaining balance auto-updates.
4	Review	Full summary of all items and payment. Edit button returns to relevant step. Confirm submits. Shows generated order number on success.

9.3 Status Color System (Consistent Everywhere)

Status	Badge Color	Tailwind Classes
NEW	Blue	bg-blue-100 text-blue-800 border-blue-200
IN_PROGRESS	Amber	bg-amber-100 text-amber-800 border-amber-200
READY	Teal	bg-teal-100 text-teal-800 border-teal-200
OVERDUE	Red	bg-red-100 text-red-800 border-red-200 font-bold
DELIVERED	Purple	bg-purple-100 text-purple-800 border-purple-200
COMPLETED	Green	bg-green-100 text-green-800 border-green-200
CANCELLED	Gray	bg-gray-100 text-gray-500 border-gray-200 line-through

9.4 Key Utility Functions

```
// lib/utils.ts — shared across entire frontend
```

```
// ALWAYS use this for displaying money
export function formatPKR(paisas: number): string {
  return `Rs. ${(paisas / 100).toLocaleString('en-PK', { minimumFractionDigits: 0 })}`;
}

// Date display — DD/MM/YYYY format
export function formatDate(date: string | Date): string {
  return new Date(date).toLocaleDateString('en-PK', {
    day: '2-digit', month: '2-digit', year: 'numeric', timeZone: 'Asia/Karachi'
  });
}

// Order number generator (called on backend, shown on frontend)
// Format: ORD-2025-LHR-00047

// Size number (auto on backend, shown on customer card)
// Format: C-LHR-0045
```

10. Phased Delivery Plan

Each phase is independently deployable. Build in this order. Use this PRD as Cursor context per phase.

Phase 1 — Foundation (Weeks 1–5)

Deliverable: Working app with auth, employee management, customer management, and order tracking.

- Monorepo setup: Next.js 14 + NestJS + Prisma + PostgreSQL (Neon) + Redis (Upstash) + Docker
- Full Prisma schema with all migrations + FTS index SQL
- Authentication: JWT login/refresh/logout, RBAC guards, BranchGuard, AuditInterceptor
- Admin Config: garment types (customerPrice + employeeRate) + branch price overrides
- Admin Config: measurement categories + dynamic fields
- Employee Module: full CRUD + document uploads to R2 + FTS search
- Customer Module: full CRUD + dynamic measurements + sizeNumber auto-gen + FTS search
- Order Module: create orders + add items (price snapshot) + assign employees
- Order status transitions with validation rules + OrderStatusHistory logging
- Overdue cron job (@Cron hourly) — auto-sets OVERDUE status
- Discount system: FIXED and PERCENTAGE on orders
- Employee login (EMPLOYEE role) with /my/ endpoints
- GitHub Actions CI/CD pipeline

Phase 2 — Payments & Finance (Weeks 6–9)

Deliverable: Complete financial layer — employee disbursements, customer payments, expenses.

- Employee Payment Module: balance calc + weekly breakdown (PKT timezone) + disbursement
- Customer payment recording per order (advance + subsequent payments)
- Business Expense Tracker with admin-configurable categories + receipt upload
- Basic Dashboard: KPI cards (revenue, expenses, outstanding employee balances, overdue orders)
- PDF receipt generation per order (@react-pdf)
- Order balance due calculation with discount support

Phase 3 — Analytics & Reports (Weeks 10–13)

Deliverable: Full analytics, exportable reports, polished UI.

- Full dashboard charts: revenue vs expenses (bar), orders by garment type (pie), employee productivity (bar)
- Weekly payment report — printable summary of all employees
- Excel + PDF exports for: orders, payments, expenses, employee summaries
- Multi-branch dashboard: Super Admin sees all branches aggregated + per-branch drill-down
- Dashboard overdue orders widget with quick-link to filtered order list
- UI polish: mobile-responsive layout, loading skeletons, empty states

Phase 4 — Future Scale (Post-Launch)

Not in initial scope — triggered by business growth.

- Fabric / raw material inventory tracking
- WhatsApp Business API — order ready notifications
- Customer order status page (shareable link + PIN)
- Attendance / clock-in tracking for employees
- Customer loyalty tracking (VIP flag, lifetime value)
- Multi-schema PostgreSQL (if 10+ branches)

11. Testing Requirements

Type	Tool	Target	Critical Tests
Unit	Jest	80% coverage	Payment calculations, price resolution, discount math, order total recalc
Integration	Jest + Supertest	All endpoints	Auth guards (401/403), branch isolation, RBAC rules, status transitions
E2E	Playwright	Happy paths	Login → create customer → create order → mark complete → disburse payment

11.1 Must-Pass Test Cases

- Payment: disbursement > balance → 422 PAYMENT_EXCEEDS_BALANCE
- Payment: cancelled item not included in employee earnings
- Payment: OVERDUE order items still earn pay if COMPLETED before cancel
- Branch: user from Branch A cannot access Branch B customers (403)
- RBAC: ENTRY_OPERATOR cannot apply discount (403)
- RBAC: EMPLOYEE cannot see customer name/phone on /my/orders
- Discount FIXED: never reduces total below 0
- Discount PERCENTAGE: floor division (no floating point)
- Order total recalc: cancelled item excluded from subtotal
- Overdue cron: only transitions NEW/IN_PROGRESS/READY — not DELIVERED/COMPLETED/CANCELLED
- Size number: unique constraint enforced, correct branch code prefix
- Price snapshot: changing garment type price does not alter existing OrderItems

12. Cursor / AI Agent Prompts

Use these prompt templates when starting each Cursor session. Always attach this full PRD as context.

Phase 1 — Session A: Project Setup & Schema

Cursor Prompt

You are building TBMS (Tailoring Business Management System) for My Tailor & Fabrics.
The full PRD (v2.1 FINAL) is attached as context.

Task: Set up the monorepo (Section 2.3) and create the complete Prisma schema (Section 3).
- Create the folder structure exactly as defined in Section 2.3
- Implement the full prisma/schema.prisma from Section 3
- Write the FTS index SQL migrations from Section 4.1
- Set up docker-compose.yml for local dev (postgres + redis)
- Configure the .env.example from Section 8.4
- Set up Prisma with DIRECT_URL + DATABASE_URL for Neon

Follow all conventions in Section 3 (paisas for money, soft deletes, branch_id on all tables).

Phase 1 — Session B: Authentication & Guards

Cursor Prompt

Context: TBMS PRD v2.1 attached. Schema is complete (Phase 1A done).

Task: Implement the full authentication system (Section 6 of PRD).
- JWT strategy with 15min access + 7d HttpOnly refresh token
- bcrypt cost factor 12 for password hashing
- All 5 roles: SUPER_ADMIN, ADMIN, ENTRY_OPERATOR, VIEWER, EMPLOYEE
- RolesGuard with hierarchy function (Section 6 of PRD)
- BranchGuard that injects req.branchId (Section 6 of PRD)
- AuditInterceptor that logs all POST/PUT/PATCH/DELETE
- /auth/login, /auth/refresh, /auth/logout, /auth/me endpoints
- Employee self-service /my/* endpoints (EMPLOYEE role, own data only)

Employee role must NEVER see customer prices (unitPrice). Only employeeRate.

Phase 1 — Session C: Search & Core Modules

Cursor Prompt

Context: TBMS PRD v2.1 attached. Auth is complete (Phase 1B done).

Task: Implement SearchService, CustomerModule, EmployeeModule.
- SearchService (Section 4.2): FTS with Redis cache, size-number fast path, 250ms debounce hook

- CustomerModule: full CRUD, sizeNumber auto-gen (C-{branchCode}-{4digits}), dynamic measurements
- EmployeeModule: full CRUD, R2 document uploads, employee login account creation
- All endpoints from Section 6.2 for /customers, /employees, /search
- Apply branch scoping on all queries (deletedAt IS NULL + branchId filter)

Phase 1 — Session D: Orders & Scheduler

Cursor Prompt

Context: TBMS PRD v2.1 attached. Customers + Employees are complete (Phase 1C done).

Task: Implement OrderModule + OverdueScheduler.
- Full order CRUD from Section 6.2
- Price resolution service (Decision 2 / Section 5.1): branch override → global fallback
- Price snapshot on OrderItem creation (unitPrice + employeeRate + garmentTypeName)
- recalcOrderTotals() from Section 5.2 — called after every item add/update/cancel
- Discount system: FIXED + PERCENTAGE (Section 5.2)
- Order status transitions with validation rules (Section 5.6)
- OrderStatusHistory logging for every transition
- OverdueScheduler @Cron(EVERY_HOUR) from Section 5.5
- PDF receipt endpoint using @react-pdf

Phase 2 — Payments Session

Cursor Prompt

Context: TBMS PRD v2.1 attached. Phase 1 fully complete.

Task: Implement PaymentModule + ExpenseModule.
- getEmployeeBalanceSummary() with raw SQL from Section 5.3
- getWeeklyBreakdown() with PKT timezone from Section 5.4
- disbursePay() with PAYMENT_EXCEEDS_BALANCE guard from Section 5.3
- Customer order payment recording + balanceDue recalc on Order
- Cancelled items must NEVER appear in employee earnings query
- ExpenseModule: full CRUD with R2 receipt upload, admin-configurable categories
- Dashboard endpoint: KPI cards + chart data (revenue, expenses, outstanding balances)